

BDCC Midterm Reviewer: Frequent Itemsets & Recommender Systems

Sep 28, 2026 · @Marianne

Midterm game plan

Exam part	Weight	What it tests	Study with
Multiple choice	60%	Definitions, which method fits when, reading a formula, interpreting lift or NDCG	Parts 1–7, then MCQ practice
Hands-on in Jupyter	40%	Writing the exercise functions so their asserts pass	The review notebook BDCC_Midterm_Review.ipynb , and Jupyter part below

Study loop for each topic (by hand first)

1. **Read** the part below and follow the slide example line by line.
2. **Redo it on paper** without looking, then try the matching drill in **By-hand drills**.
3. **Check yourself** in the notebook's  cells: type your paper answers, run, and get  or  per number.
4. **Code it** in the notebook's  drills, run the  tests, and only then open  the solution.

Last day before the exam

- Do **MCQ practice** cold, then reread only the parts you missed.
- On the course JupyterHub, run notebook **Part D**. It reruns the original Jester tests on your own functions.
- Skim the notebook's **Part E** cheat sheet (sorting ties, `.loc` vs `.iloc`, masks).

Start here

This whole topic answers two everyday questions. First: **what do people buy together?** (Parts 1–3). Second: **what will this person like next?** (Parts 4–7).

Think of a grocery store and a movie app.

- The grocery store looks at receipts and notices "people who buy bread also buy milk." That is **frequent itemset mining** and **association rules**.
- The movie app looks at star ratings and guesses the stars you'd give a movie you haven't seen. That is a **recommender system**.

Every idea below has three parts: the plain-English meaning, the formula, and a tiny example you can check by hand.

Symbol guide (read this once)

Symbol	Say it as	Means
N	"N"	How many receipts (transactions) there are
X, A, B	"X"	A group of items, like {bread, milk}
U	"union" / "together with"	Put two groups together: {bread} U {milk} = {bread, milk}
$\subseteq$	"is inside"	Every item of the left group is in the right group
Σ	"sigma" / "add up"	Add up everything that follows
r_{ui}	"r of u, i"	The rating user u gave item i
μ_u	"mu of u"	User u 's average rating
$\hat{r}_{ui}$	"r-hat"	Our guess of user u 's rating for item i
$\text{sim}(u, v)$	"similarity"	How alike two users (or two items) are, from -1 to 1
k	"k"	How many look-alikes (neighbors) we use
U, V	"U", "V"	Hidden-taste tables for users and for items

The course material behind this: the three notebooks by Christian Alis and the BDCC slides for Sessions 3–4, 5–7 and 8–10.

Part 1: What goes together?

We count how many receipts contain a group of items. Groups that show up often enough are called **frequent**.

Our 5 receipts (used all the way through Parts 1–3)

Receipt	Items bought
1	bread, butter, milk
2	eggs, milk, yogurt
3	bread, cheese, eggs, milk
4	eggs, milk, yogurt
5	cheese, milk, yogurt

Words you need

- **Transaction** = one receipt. We have $N = 5$.
- **Item** = one product (bread, milk...). There are 6 different items here.
- **Itemset** = any group of items. A group of 2 items is a **2-itemset**, of 3 items a **3-itemset**, and so on.

One receipt hides many small groups. Receipt 1 {bread, butter, milk} contains 7 groups: 3 singles, 3 pairs, and 1 triple. In general a set of k items has this many non-empty groups:

$$2^k - 1$$

For $k = 3$: $2^3 - 1 = 8 - 1 = 7$. ✓

Support = "how many receipts have it?"

Absolute support is a plain count:

$$\text{sup}(X) = \text{number of receipts that contain every item in } X$$

Relative support turns that count into a fraction of all receipts:

$$\text{relSup}(X) = \frac{\text{sup}(X)}{N}$$

Example: bread and milk appear together on receipts 1 and 3.

$$\text{sup}(\{\text{bread, milk}\}) = 2 \quad \text{relSup}(\{\text{bread, milk}\}) = \frac{2}{5} = 0.4$$

Frequent = "shows up at least minsup times"

You pick a cut-off called **minsup** (minimum support). A group is frequent if:

$$\text{sup}(X) \geq \text{minsup}$$

Slide example, using the groups inside receipt 1:

Group	sup	relSup
milk	5	1.0
bread	2	0.4
bread, milk	2	0.4
butter	1	0.2
bread, butter	1	0.2
butter, milk	1	0.2
bread, butter, milk	1	0.2

- minsup = 3 (or 0.5 as a fraction) → only {milk} passes.
- minsup = 2 (or 0.4) → {milk}, {bread}, and {bread, milk} pass.

The one rule that makes everything faster

Adding items can never make a group more common. A pair can't appear on more receipts than either of its items alone.

$$Y \subseteq X \implies \text{sup}(Y) \geq \text{sup}(X)$$

Read it as: "if Y is part of X, then Y is at least as common as X." Two handy results:

- **Downward closure:** every part of a frequent group is also frequent.

- **The flip side:** if a group is NOT frequent, nothing bigger that contains it can be frequent. Butter appears once, so any group with butter is out.

Maximal frequent itemset = a frequent group you can't make any bigger without it failing.

All frequent groups at minsup = 2

Size	Frequent groups (support)
1 item	milk (5), eggs (3), yogurt (3), bread (2), cheese (2)
2 items	eggs+milk (3), milk+yogurt (3), eggs+yogurt (2), bread+milk (2), cheese+milk (2)
3 items	eggs+milk+yogurt (2)

That's 11 frequent groups. The maximal ones are {eggs, milk, yogurt}, {bread, milk}, and {cheese, milk}.

Part 2: Three ways to find the frequent groups

All three methods give the same 11 answers. They differ in how much work they waste.

Method	Idea in one line	Slide verdict
Brute force	Check every possible group	Simple but very slow
Apriori	Only grow groups that already passed	Faster, but more complicated
FP-Growth	Shrink the receipts as you go	Parallelizable, but hardest to code

1. Brute force: check everything

List every possible group of the d items, count each one on every receipt, and keep the ones that pass.

The number of groups to check doubles with every new item:

$$\text{groups to check} = 2^d - 1$$

So the total work is roughly:

$$\text{work} \approx N \times 2^d$$

- Our data: $d = 6$ items, $N = 5$ receipts $\rightarrow 63$ groups $\times 5$ receipts = 315 checks. Easy.
- 1,000 items $\rightarrow$ about 1.7×10^{299} times more work. Impossible.
- 1,000 receipts (still 6 items) $\rightarrow$ only 200 times more work. Fine.

Lesson: more items is the killer (exponential). More receipts just adds work in a straight line (linear).

2. Apriori: only grow winners

Use the rule from Part 1: if a group fails, anything bigger that contains it fails too. So we build up one size at a time.

1. Count all single items. Keep the frequent ones.
2. Combine the winners into pairs. Count them. Keep the frequent ones.
3. Combine winning pairs into triples, but skip any triple that has a failed pair inside it.
4. Keep going until nothing new passes.

Slide walkthrough, minsup = 2:

Round	What gets counted (support)	What passes
Singles	bread 2, butter 1, milk 5, eggs 3, yogurt 3, cheese 2	everything except butter
Pairs	bread+milk 2, bread+eggs 1, bread+yogurt 0, bread+cheese 1, milk+eggs 3, milk+yogurt 3, milk+cheese 2, eggs+yogurt 2, eggs+cheese 1, yogurt+cheese 1	bread+milk, milk+eggs, milk+yogurt, milk+cheese, eggs+yogurt
Triples	milk+eggs+yogurt 2 (others 0 or 1)	milk+eggs+yogurt

Butter failed in round 1, so no group with butter was ever counted. That's the saving.

Weak spots: Apriori still invents groups that never appear on any receipt, and it re-reads all receipts every round.

3. FP-Growth: shrink the receipts

Pick one item. Keep only the receipts that have it, and cross that item off. This smaller pile is called the **projected database**. Look for frequent items in the small pile, and repeat.

Slide walkthrough, minsup = 2, items in alphabetical order:

Starting group	Small pile left over	What we find
bread	R1 {butter, milk}, R3 {cheese, eggs, milk}	bread (2)
bread + milk	R1 { }, R3 { }	bread+milk (2)
cheese	R3 {eggs, milk}, R5 {milk, yogurt}	cheese (2), cheese+milk (2)
eggs	R2 {milk, yogurt}, R3 {milk}, R4 {milk, yogurt}	eggs (3), eggs+milk (3), eggs+milk+yogurt (2), eggs+yogurt (2)
milk	all 5 receipts, only yogurt left	milk (5), milk+yogurt (3)
yogurt	R2, R4, R5, nothing left	yogurt (3)

Butter and pairs like bread+eggs only appear once in their pile, so they are dropped.

The coded version packs the receipts into a tree where shared starts share branches (the **FP-tree**). It only reads the receipts twice.

Speed test from the notebook

FP-Growth won both times. With more items, Apriori went from 0.043 s to 0.515 s while FP-Growth stayed around 0.01 s. With more receipts, both grew steadily, but FP-Growth stayed about 10 times faster.

Part 3: Rules — "if they buy A, they'll buy B"

A rule is written $A \rightarrow B$. A is the **antecedent** (what's already in the cart). B is the **consequent** (what we predict gets added).

Confidence = "how often does the rule come true?"

Out of the receipts that have A, what fraction also have B?

$$\text{conf}(A \rightarrow B) = \frac{\text{sup}(A \cup B)}{\text{sup}(A)}$$

Example: does buying yogurt lead to buying milk?

$$\text{conf}(\text{yogurt} \rightarrow \text{milk}) = \frac{\text{sup}(\{\text{yogurt, milk}\})}{\text{sup}(\{\text{yogurt}\})} = \frac{3}{3} = 1.0$$

Flip it around and you get a different answer. Direction matters:

$$\text{conf}(\text{milk} \rightarrow \text{yogurt}) = \frac{3}{5} = 0.6$$

Lift = "does A actually help, or is B just popular?"

Confidence can fool you. Milk is on every receipt, so "anything $\rightarrow$ milk" always looks great. Lift compares the rule to how common B already is:

$$\text{lift}(A \rightarrow B) = \frac{\text{conf}(A \rightarrow B)}{\text{relSup}(B)}$$

Lift	Meaning	Everyday version
Bigger than 1	A and B go together	"Buying A makes B more likely"
Exactly 1	No connection	"A doesn't matter for B"
Less than 1	A and B avoid each other	"Buying A makes B less likely"

Slide example 1: bread $\rightarrow$ butter

$$\text{conf} = \frac{1}{2} = 0.5 \quad \text{relSup}(\text{butter}) = \frac{1}{5} = 0.2 \quad \text{lift} = \frac{0.5}{0.2} = 2.5$$

Lift 2.5 $\rightarrow$ bread buyers are 2.5 times more likely to get butter than an average shopper.

Slide example 2: bread $\rightarrow$ eggs

$$\text{conf} = \frac{1}{2} = 0.5 \quad \text{relSup}(\text{eggs}) = \frac{3}{5} = 0.6 \quad \text{lift} = \frac{0.5}{0.6} = 0.83$$

Lift below 1 $\rightarrow$ bread buyers are slightly *less* likely to buy eggs.

Trap example: bread $\rightarrow$ milk. Confidence is $2/2 = 1.0$, which sounds perfect. But $\text{relSup}(\text{milk}) = 5/5 = 1$, so $\text{lift} = 1.0$: no real connection. Everyone buys milk anyway.

How to find all the rules (slide method)

1. Find all frequent groups (Part 2).
2. Split each group of 2+ items into every possible "left side $\rightarrow$ right side."
3. Compute the confidence of each. Keep the ones at or above **minconf**.

The notebook's code does a narrower version: the right side is always a single item. The slides allow more than one item on the right, like $\text{eggs} \rightarrow \{\text{yogurt}, \text{milk}\}$.

Slide answer: $\text{minsup} = 2$, $\text{minconf} = 0.6 \rightarrow 13$ rules

Rule	Confidence
$\text{yogurt} \rightarrow \text{milk}$	$3/3 = 1.00$
$\text{eggs} \rightarrow \text{milk}$	$3/3 = 1.00$
$\{\text{eggs}, \text{yogurt}\} \rightarrow \text{milk}$	$2/2 = 1.00$
$\text{bread} \rightarrow \text{milk}$	$2/2 = 1.00$
$\text{cheese} \rightarrow \text{milk}$	$2/2 = 1.00$
$\{\text{eggs}, \text{milk}\} \rightarrow \text{yogurt}$	$2/3 = 0.67$
$\{\text{yogurt}, \text{milk}\} \rightarrow \text{eggs}$	$2/3 = 0.67$
$\text{eggs} \rightarrow \{\text{yogurt}, \text{milk}\}$	$2/3 = 0.67$
$\text{yogurt} \rightarrow \{\text{eggs}, \text{milk}\}$	$2/3 = 0.67$
$\text{eggs} \rightarrow \text{yogurt}$	$2/3 = 0.67$
$\text{yogurt} \rightarrow \text{eggs}$	$2/3 = 0.67$
$\text{milk} \rightarrow \text{yogurt}$	$3/5 = 0.60$
$\text{milk} \rightarrow \text{eggs}$	$3/5 = 0.60$

Thrown out for being under 0.6: $\text{milk} \rightarrow \text{bread}$, $\text{milk} \rightarrow \text{cheese}$, and $\text{milk} \rightarrow \{\text{eggs}, \text{yogurt}\}$ (each $2/5 = 0.4$).

Real-world use (notebook's online store)

After cleaning the data (removing cancelled orders and fees), the store had 19,963 receipts. The rules suggested three things: sell matching tea sets as bundles, stock color pairs together (pink and blue cups sell as a pair), and build ready-made party and Christmas packs.

Part 4: Recommenders — filling in the blanks

A recommender has a big table of who rated what. Most of it is empty. Its job is to **guess the empty cells**, then show each person the items with the highest guesses.

The ratings table (called the utility matrix)

Rows are people, columns are movies (HP = Harry Potter, TW = Twilight, SW = Star Wars). Stars go from 1 to 5.

User	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D	3				3		

(This follows the slides. The notebook's code puts D's two 3s under HP2 and SW3 instead; it doesn't change any answer, because D gave the same score to everything.)

In real apps the table is mostly empty: there can be 100,000+ users and items, but each person rates only about 10.

Two kinds of ratings

- **Explicit:** the person tells you ("4 stars").
- **Implicit:** you infer it from behavior (they watched it, bought it, clicked it). Stored as 1 or blank. Easier to collect, so the table fills up more.

A good recommendation is...

- **Relevant:** something they'd actually like.
- **Novel:** something they haven't seen.

- **Serendipitous:** a pleasant surprise, not an obvious pick.
- **Diverse:** the list isn't ten copies of the same thing.

Two families of recommenders

Type	Looks at	Plain example
Content-based	What the items are like	"You like red, round things, so here are more red, round things."
Collaborative filtering	Only the ratings from many people	"People with your taste liked this."

Collaborative filtering (CF) itself comes in two flavors:

1. **Neighborhood-based (memory-based)** — find look-alike people or items. That's Part 5.
2. **Model-based (latent factors)** — learn hidden tastes behind the ratings. That's Part 7.

The coding exercises use the **Jester** jokes data: 100 users × 100 jokes, ratings from −10 to 10, and 99 means "not rated."

Part 5: Finding look-alikes (neighborhood CF)

To guess your rating, find people (or movies) that are similar, then take a weighted average of their ratings. More similar = more say in the answer.

Step 1: Remove each person's "grading style" (mean-centering)

Some people give everyone 5 stars; others are harsh. So first subtract each person's own average from their ratings. What's left shows whether they liked something **more or less than usual**.

$$\text{centered rating} = r_{ui} - \mu_u$$

Slide example (HP/TW/SW table):

User	Average μ	Centered ratings
A	$10/3 \approx 3.33$	HP1 +2/3, TW +5/3, SW1 −7/3
B	$14/3 \approx 4.67$	HP1 +1/3, HP2 +1/3, HP3 −2/3

User	Average μ	Centered ratings
C	$11/3 \approx 3.67$	TW $-5/3$, SW1 $+1/3$, SW2 $+4/3$
D	3	HP1 0, SW1 0

Step 2: Measure how alike two people are

Only compare movies both people rated. Never fill blanks with a made-up number.

Raw cosine (the simple version) multiplies matching ratings and scales the result to between 0 and 1:

$$\text{RawCosine}(u, v) = \frac{\sum_i r_{ui} r_{vi}}{\sqrt{\sum_i r_{ui}^2} \sqrt{\sum_i r_{vi}^2}}$$

The problem: A and C rated TW and SW1 oppositely (A: 5, 1; C: 2, 4), yet raw cosine calls them fairly similar:

$$\text{RawCosine}(A, C) = \frac{5 \cdot 2 + 1 \cdot 4}{\sqrt{5^2 + 1^2} \sqrt{2^2 + 4^2}} = \frac{14}{\sqrt{26} \sqrt{20}} \approx 0.61$$

Adjusted cosine does the same thing on the centered ratings. This is the one the course uses. It gives a number from -1 (opposite taste) to $+1$ (same taste). It is the same as the Pearson correlation.

$$\text{sim}(u, v) = \frac{\sum_i (r_{ui} - \mu_u)(r_{vi} - \mu_v)}{\sqrt{\sum_i (r_{ui} - \mu_u)^2} \sqrt{\sum_i (r_{vi} - \mu_v)^2}}$$

Now A and C come out as opposites, which matches common sense:

$$\text{sim}(A, C) = \frac{\left(\frac{5}{3}\right)\left(-\frac{5}{3}\right) + \left(-\frac{7}{3}\right)\left(\frac{1}{3}\right)}{\sqrt{\left(\frac{5}{3}\right)^2 + \left(-\frac{7}{3}\right)^2} \sqrt{\left(-\frac{5}{3}\right)^2 + \left(\frac{1}{3}\right)^2}} \approx -0.73$$

Slide results: $\text{sim}(A, B) = 1$, $\text{sim}(A, C) = -0.73$, and every similarity with D is 0 (D's centered ratings are all 0, so there's nothing to compare).

Step 3a: User-based guess — "people like you rated it..."

Take the k most similar people who rated that item. Average their centered ratings, weighted by similarity. Then add back your own average.

$$\hat{r}_{ui} = \mu_u + \frac{\sum_{v \text{ sim}(u,v)} (r_{vi} - \mu_v)}{\sum_{v \text{ sim}(u,v)}}$$

In words: **your average + (weighted average of how much your look-alikes liked it more or less than usual).**

In all the slide examples, only look-alikes with a **positive** similarity are used; people with opposite taste are left out.

Heads-up for the Jupyter part: the exercise code does it differently. It takes the top k by similarity (negative ones included), then keeps those who rated the item. On the slides' table this gives a different number for B's rating of c (7.35 instead of 6.0). With Jester and $k = 5$, the top neighbors are usually all positive, so the two rules tend to agree there. Follow whatever rule the question states.

Worked example (slides): 5 users, 6 items, $k = 3$

User	a	b	c	d	e	f	Average
A	7	6	7	4	5	4	5.5
B	6	7	?	4	3	4	4.8
C	?	3	3	1	1	?	2
D	1	2	2	3	3	4	2.5
E	1	?	1	2	3	3	2

User similarities:

	A	B	C	D	E
A	—	0.70	0.89	-0.90	-0.82
B	0.70	—	0.94	-0.72	-0.90
C	0.89	0.94	—	-1.00	-0.82
D	-0.90	-0.72	-1.00	—	0.87
E	-0.82	-0.90	-0.82	0.87	—

Guess B's rating for c. B's positive look-alikes who rated c: A (sim 0.70, centered 7 – 5.5 = 1.5) and C (sim 0.94, centered 3 – 2 = 1).

$$\hat{r}_{B,c} = 4.8 + \frac{0.70(1.5) + 0.94(1)}{0.70 + 0.94} = 4.8 + \frac{1.99}{1.64} = 4.8 + 1.2 = 6.0$$

The other three blanks, the same way:

Blank	Look-alikes used	Centered guess	Final guess
C, a	A (0.89, +1.5), B (0.94, +1.2)	+1.3	2 + 1.3 = 3.3
C, f	A (0.89, –1.5), B (0.94, –0.8)	–1.1	2 – 1.1 = 0.9
E, b	D (0.87, –0.5)	–0.5	2 – 0.5 = 1.5

Note: the slides write the centered guess (like 1.2) in the table. Add the person's average to get the real star rating.

Step 3b: Item-based guess — "you rated similar items..."

Same idea, flipped. Find the k items most similar to the one we're guessing, among items **you** rated. Average your own centered ratings on them.

$$\hat{r}_{ui} = \mu_u + \frac{\sum_j \text{sim}(i,j) (r_{uj} - \mu_u)}{\sum_j \text{sim}(i,j)}$$

Item similarity uses the same adjusted cosine, but compares two **columns** (still centered by each person's average).

Slide example, 6-item table, k = 2. Item similarities:

	a	b	c	d	e	f
a	—	0.74	0.91	–0.85	–0.81	–0.99
b	0.74	—	0.87	–0.73	–1.00	–0.62
c	0.91	0.87	—	–0.88	–0.89	–0.91
d	–0.85	–0.73	–0.88	—	0.71	0.83
e	–0.81	–1.00	–0.89	0.71	—	0.73
f	–0.99	–0.62	–0.91	0.83	0.73	—

Guess B's rating for c. The 2 items most like c are a (0.91) and b (0.87). B's centered ratings: $a = 6 - 4.8 = 1.2$, $b = 7 - 4.8 = 2.2$.

$$\hat{r}_{B,c} = 4.8 + \frac{0.91(1.2) + 0.87(2.2)}{0.91 + 0.87} = 4.8 + 1.7 = 6.5$$

Blank	Two most similar items (your centered rating)	Centered guess
C, a	c 0.91 (+1), b 0.74 (+1)	+1.0
E, b	c 0.87 (-1), a 0.74 (-1)	-1.0
C, f	d 0.83 (-1), e 0.73 (-1)	-1.0

The slides also run item-based on the HP/TW/SW table ($k = 2$). For example, C's HP1 guess uses TW (similarity 1, C's centered TW = $-5/3$), giving -1.7 . A's HP2 uses HP1 (similarity 1, centered $+2/3$), giving $+0.7$.

User-based vs item-based

	User-based	Item-based
Looks for	Similar people	Similar items
Getting to a recommendation	Direct	One extra step
How reliable	Weaker: people like many genres	Stronger: items group neatly by genre

Final step either way: sort your unrated items by guessed rating, highest first (ties broken by id), and show the top n .

Part 6: Grading the recommendations

There are two ways to grade: "were the guessed stars close?" and "was the best stuff at the top of the list?"

Online vs offline testing

- **Online:** real people use the live system (for example, an A/B test). Realistic, but costly.

- **Offline:** test on ratings you already collected. Cheap and common, but the data can be old.

What we could grade

Quality	Question it asks
Accuracy	Are the guesses right? (the only one graded in this course)
Coverage	How many users can we give recommendations to?
Confidence and trust	How sure are we, and does the user believe us?
Novelty	Is it something they didn't know?
Serendipity	Was it a pleasant surprise?
Diversity	Is the list varied?
Robustness and stability	Can it resist fake ratings and changing habits?
Scalability	Does it still work with huge data?

Grade 1: "Were the stars close?"

Treat it like any prediction. The most common scores:

$$\text{MAE} = \frac{1}{n} \sum \left| r - \hat{r} \right| \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum \left(r - \hat{r} \right)^2}$$

MAE = the average miss. RMSE punishes big misses more. Smaller is better for both.

Grade 2: "Was the good stuff on top?" (DCG)

People mostly look at the first few items, so a great item at #1 should count more than one at #5.

Each item earns points (**gain**) from its true rating, then gets shrunk (**discounted**) the further down the list it sits:

$$\text{gain} = 2^r - 1 \quad \text{discount} = \log_2(\text{position} + 1)$$

$$\text{DCG} = \sum_{\text{position}=1}^L \frac{2^{r_{\text{position}}} - 1}{\log_2(\text{position} + 1)}$$

For many users, the notebook averages DCG over the m users:

$$\text{DCG} = \frac{1}{m} \sum_{u=1}^m \sum_j \frac{2^{r_{uj}} - 1}{\log_2(v_j + 1)}$$

NDCG compares your list to the best possible list (**IDCG**, the ideal order). It's always between 0 and 1; closer to 1 is better.

$$\text{NDCG} = \frac{\text{DCG}}{\text{IDCG}}$$

Worked example (slides)

True ratings: A = 9, B = 3, C = 7, D = 2, E = 5, F = 8. The system recommended **[B, A, C, F, E]**. The best possible top 5 is **[A, F, C, E, B]**.

Position	Recommended (rating)	Points $2^r - 1$	Shrink by $\log_2(\text{pos}+1)$	Ideal (rating)	Ideal points
1	B (3)	7	1.000	A (9)	511
2	A (9)	511	1.585	F (8)	255
3	C (7)	127	2.000	C (7)	127
4	F (8)	255	2.322	E (5)	31
5	E (5)	31	2.585	B (3)	7

$$\text{DCG} = \frac{7}{1} + \frac{511}{1.585} + \frac{127}{2} + \frac{255}{2.322} + \frac{31}{2.585} = 514.72$$

$$\text{IDCG} = \frac{511}{1} + \frac{255}{1.585} + \frac{127}{2} + \frac{31}{2.322} + \frac{7}{2.585} = 751.45$$

$$\text{NDCG} = \frac{514.72}{751.45} = 0.685$$

Why so low? The best item (A, rated 9) was put second instead of first. Because points grow exponentially, that one slip costs the most.

In the code, a recommended item the user never rated counts as 0 points ($2^0 - 1 = 0$).

Part 7: Hidden tastes (latent factor CF)

Imagine every movie has hidden scores, like "how much action" and "how much romance." Every person has matching scores for how much they enjoy each. Multiply and add them up, and you get a predicted rating. We never see these hidden scores; the computer figures them out from the ratings.

The big idea: split the table into two smaller tables

$$\mathbf{M} \approx \mathbf{U}\mathbf{V}^T$$

- **M** = the ratings table: n people $\times$ k items.
- **U** = people's hidden tastes: n people $\times$ d tastes.
- **V** = items' hidden traits: k items $\times$ d traits.
- **d** = how many hidden factors we allow (we choose it).

One rating is just one person's row times one item's row:

$$\hat{r}_{ij} = \sum_{s=1}^d u_{is} v_{js} = u_{i1}v_{j1} + u_{i2}v_{j2} + \dots + u_{id}v_{jd}$$

Bonus: storing **U** and **V** takes $(n + k) \times d$ numbers instead of $n \times k$. For 1,000 people, 500 items and $d = 20$, that's 30,000 instead of 500,000.

What "good" means: small squared error on the ratings we know

$$J = \frac{1}{2} \sum_{(i,j) \in \text{known}} \left(r_{ij} - \sum_{s=1}^d u_{is} v_{js} \right)^2$$

We only count cells that actually have a rating. Blanks are what we want to predict, so they're left out.

Method 0: Gradient descent (the general idea)

Nudge **U** and **V** a little in the direction that lowers the error, again and again. With **E** = the table of errors on known cells, and α = step size:

$$\begin{aligned} \mathbf{U} &\leftarrow \mathbf{U} + \alpha \mathbf{E} \mathbf{V} \\ \mathbf{V} &\leftarrow \mathbf{V} + \alpha \mathbf{E}^T \mathbf{U} \end{aligned}$$

- **Batch** gradient descent uses every known rating each step.

- **Stochastic** gradient descent (SGD) uses a random sample each step. Faster, but bumpier.

Method 1: Alternating Least Squares (ALS)

Freeze one table, solve the other exactly, then swap. Repeat until the error is small enough.

1. Freeze U. Solve for V (one small regression per item).
2. Freeze V. Solve for U (one small regression per person).
3. Repeat.

Each small regression is independent, so they can all run at the same time (parallel). ALS is steadier than SGD.

Slide example. The ratings table (5 people $\times$ 5 items; blanks are unknown):

```
\mathbf{M} = \begin{pmatrix} 5 & 2 & 4 & 4 & 3 & \backslash & 3 & 1 & 2 & 4 & 1 & \backslash & \cdot \\ & \cdot & 3 & 1 & 4 & \backslash & 2 & 5 & 4 & 3 & 5 & \backslash & 4 & 4 & 5 & 4 & \cdot \\ & & & & & & & & & & & & & & & \end{pmatrix}
```

Start with $d = 2$ and every $u = 1$. Then each prediction in column 1 is just $v_{11} + v_{12}$. We want that sum as close as possible to the known ratings 5, 3, 2, 4:

```
\min \; [5 - (v_{11}+v_{12})]^2 + [3 - (v_{11}+v_{12})]^2 + [2 - (v_{11}+v_{12})]^2 + [4 - (v_{11}+v_{12})]^2
```

The best single number to match several numbers is their average: $(5 + 3 + 2 + 4)/4 = 3.5$. Split evenly, $v_{11} = v_{12} = 1.75$. Doing every column:

```
\mathbf{V}^T = \begin{pmatrix} 1.75 & 1.5 & 1.8 & 1.6 & 1.625 & \backslash & 1.75 & 1.5 \\ & 1.8 & 1.6 & 1.625 & \end{pmatrix}
```

Now freeze V and solve U the same way. The slides get:

```
\mathbf{U} = \begin{pmatrix} 1.099 & 1.099 & \backslash & 0.668 & 0.668 & \backslash & 0.800 & 0.800 \\ & 1.132 & 1.132 & \backslash & 1.278 & 1.278 & \end{pmatrix}
```

Then check the error. If it's not small enough, repeat.

Method 2: Coordinate Gradient Descent (CGD)

Even smaller steps: freeze **everything except one single number**, find its best value exactly, then move to the next number.

Slide example, step 1. Start with U and V all ones, so every prediction is $1 \cdot 1 + 1 \cdot 1 = 2$. Now let the top-left number of U be unknown, x. Only row 1 changes; each prediction becomes $x + 1$. Row 1's real ratings are 5, 2, 4, 4, 3.

$$\text{SSE} = (4-x)^2 + (1-x)^2 + (3-x)^2 + (3-x)^2 + (2-x)^2$$

Set the slope (derivative) to zero to find the lowest point:

$$-2[(4-x)+(1-x)+(3-x)+(3-x)+(2-x)] = 0 \quad \Rightarrow \quad 13 - 5x = 0 \quad \Rightarrow \quad x = 2.6$$

Step 2. Put 2.6 in, and let the top-left number of V be unknown, y. Only column 1 changes. The known ratings there are 5, 3, 2, 4 (row 3 is blank):

$$\text{SSE} = (4 - 2.6y)^2 + (2-y)^2 + (1-y)^2 + (3-y)^2$$

$$-2[2.6(4-2.6y) + (2-y) + (1-y) + (3-y)] = 0 \quad \Rightarrow \quad 16.4 - 9.76y = 0 \quad \Rightarrow \quad y \approx 1.68$$

Keep going through every number until the error stops improving much.

The general formula (what the steps above are doing):

$$u_{ij} = \frac{\sum_s v_{sj} \left(r_{is} - \sum_{t \neq j} u_{it} v_{st} \right)}{\sum_s v_{sj}^2}$$

In words: take the part of each rating that the *other* factors don't already explain, and fit this one number to it. The formula for a V number is the same with U and V swapped:

$$v_{ij} = \frac{\sum_s u_{sj} \left(r_{si} - \sum_{t \neq j} u_{st} v_{it} \right)}{\sum_s u_{sj}^2}$$

Which method when?

Method	Changes per step	Good	Not so good
Gradient descent	All of U and V, a little	Simple	You must pick a step size
SGD	U and V, from a sample	Fast	Bumpy
ALS	A whole table, exactly	Steady, runs in parallel	Slower than SGD
CGD	One number, exactly	No step size needed	Lots of tiny steps

Library check (notebook)

The **Surprise** library's KNNWithMeans gave similar but not identical guesses to our hand-written code (about 0.84 correlation for user-based, 0.72 for item-based). The differences come from small choices: how neighbors are picked, Pearson vs adjusted cosine, and Surprise capping guesses at the rating limits.

Practice questions

Cover the answer, try it, then check. Use the 5 receipts from Part 1 unless a question says otherwise.

- How many receipts contain both eggs and yogurt? What's the relative support?
 - Receipts 2 and 4 $\rightarrow$ sup = 2, relSup = $2/5 = 0.4$.
- Butter appears on only 1 receipt. At minsup = 2, can {bread, butter, milk} be frequent?
 - No. If butter alone fails, any bigger group with butter fails too.
- How many groups does brute force check if there are 20 items?
 - $2^{20} - 1 = 1,048,575$.
- Why is Apriori faster than brute force?
 - It never counts a group if a smaller part of it already failed.
- Find conf(yogurt $\rightarrow$ eggs) and lift(yogurt $\rightarrow$ eggs).
 - conf = $\text{sup}(\{\text{eggs}, \text{yogurt}\}) / \text{sup}(\text{yogurt}) = 2/3 \approx 0.67$. relSup(eggs) = $3/5 = 0.6$. lift = $0.67 / 0.6 \approx 1.11 \rightarrow$ a slight positive link.

6. bread $\rightarrow$ milk has confidence 1.0. Is it a strong rule?
 - No. Lift = $1.0 / (5/5) = 1$. Everyone buys milk, so bread tells you nothing.
7. At minconf = 0.6, is milk $\rightarrow$ {eggs, yogurt} kept?
 - No. conf = $2/5 = 0.4$.
8. Why do we subtract each person's average before comparing them?
 - To remove grading style. A generous grader and a harsh grader can have the same taste.
9. Raw cosine says A and C are 0.61 similar. Adjusted cosine says -0.73. Which is right?
 - Adjusted (-0.73). A loved TW and hated SW1; C did the opposite.
10. In the 5-user table, why doesn't D (similarity -0.72 with B) help guess B's rating for c?
 - The slides only use look-alikes with positive similarity. A and C alone give +1.2, so B's guess is $4.8 + 1.2 = 6.0$.
11. Item-based, $k = 2$: guess C's rating for a.
 - Most similar items to a that C rated: c (0.91) and b (0.74). C's centered ratings on both are +1, so the centered guess is +1, and the real guess is $2 + 1 = 3$.
12. A list's true ratings are 3 then 5. The best order is 5 then 3. What's the NDCG?
 - DCG = $7/1 + 31/1.585 \approx 26.56$. IDCG = $31/1 + 7/1.585 \approx 35.42$. NDCG ≈ 0.75 .
13. In the slide DCG example, which single swap helps the most?
 - Swap B and A, putting the 9-rated item first.
14. With U fixed at all ones and $d = 2$, a column has ratings 5, 3, 2, 4. What are its two V numbers?
 - Their sum must equal the average, $3.5 \rightarrow 1.75$ each.
15. In one sentence, how is ALS different from CGD?
 - ALS solves a whole table at once; CGD solves one number at a time.
16. How many numbers do U and V hold for 1,000 people, 500 items, and $d = 20$?
 - $(1,000 + 500) \times 20 = 30,000$, compared with 500,000 for the full table.

By-hand drills (fresh numbers)

New data, so you're not just remembering the slides. Do each one on paper, then check the worked answer. The notebook has a  cell for drills 1-3 too.

Drill 1: Apriori + rules (café receipts)

Receipt	Items
1	coffee, donut, milk
2	coffee, donut
3	coffee, milk, sugar
4	donut, milk
5	coffee, donut, milk, sugar
6	coffee, sugar

Use **minsup** = 3 and **minconf** = 0.7. Find the frequent itemsets, the maximal ones, and every single-consequent rule that passes.

Worked answer

- Singles:** coffee 5, donut 4, milk 4, sugar 3. All pass.
- Pairs:** coffee+donut 3 ✓, coffee+milk 3 ✓, coffee+sugar 3 ✓, donut+milk 3 ✓, donut+sugar 1 ✗, milk+sugar 2 ✗.
- Triples:** coffee+donut+milk has all three pairs frequent, so count it: receipts 1 and 5 → 2 ✗. Coffee+donut+sugar and coffee+milk+sugar are skipped without counting, because donut+sugar and milk+sugar already failed.
- Result:** 8 frequent itemsets. The 4 frequent pairs are the maximal ones.
- Rules** (N = 6):

Rule	Confidence	Lift	Kept?
coffee → donut	$3/5 = 0.60$		✗
donut → coffee	$3/4 = 0.75$	$0.75 \div 5/6 = 0.90$	✓
coffee → milk	$3/5 = 0.60$		✗
milk → coffee	$3/4 = 0.75$	$0.75 \div 5/6 = 0.90$	✓
coffee → sugar	$3/5 = 0.60$		✗
sugar → coffee	$3/3 = 1.00$	$1.00 \div 5/6 = 1.20$	✓

Rule	Confidence	Lift	Kept?
donut $\rightarrow$ milk	$3/4 = 0.75$	$0.75 \div 4/6 = 1.125$	✓
milk $\rightarrow$ donut	$3/4 = 0.75$	$0.75 \div 4/6 = 1.125$	✓

5 rules pass. Trap: donut $\rightarrow$ coffee has 75% confidence but lift 0.9 (below 1). Coffee is just popular; donut buyers are slightly *less* likely than average to get it.

Drill 2: user-based CF ($k = 2$)

User	i1	i2	i3	i4	i5
P	5	4	?	1	2
Q	4	5	5	1	
R	1	1	2	5	4
S	5	3	4	2	1

Guess P's rating for i3.

Worked answer

- Averages:** $P = 12/4 = 3$, $Q = 15/4 = 3.75$, $R = 13/5 = 2.6$, $S = 15/5 = 3$.
- Centered:** $P = (2, 1, \cdot, -2, -1)$; $Q = (0.25, 1.25, 1.25, -2.75, \cdot)$; $R = (-1.6, -1.6, -0.6, 2.4, 1.4)$; $S = (2, 0, 1, -1, -2)$.
- Similarities**, on items both rated (i3 excluded because P didn't rate it):

$$\text{sim}(P, S) = \frac{(2)(2) + (1)(0) + (-2)(-1) + (-1)(-2)}{\sqrt{4+1+4+1} \cdot \sqrt{4+0+1+4}} = \frac{8}{\sqrt{10} \cdot 3} \approx 0.843$$

$$\text{sim}(P, Q) = \frac{(2)(0.25) + (1)(1.25) + (-2)(-2.75)}{\sqrt{4+1+4} \cdot \sqrt{0.0625+1.5625+7.5625}} = \frac{7.25}{3 \cdot 3.031} \approx 0.797$$

$\text{sim}(P, R) \approx -0.971$ (opposite taste).

- Top 2:** S (0.843) and Q (0.797). Both are positive and both rated i3, so the slide and exercise rules agree.
- Guess:** centered i3 ratings are $S = 1$ and $Q = 1.25$.

$$\hat{r}_{P,i3} = 3 + \frac{0.843(1) + 0.797(1.25)}{0.843 + 0.797} = 3 + \frac{1.839}{1.640} \approx 3 + 1.12 = 4.12$$

Drill 3: NDCG

True ratings: $W = 4, X = 1, Y = 3, Z = 2$. The system showed $[X, W, Y]$.

Worked answer

$$\text{DCG} = \frac{2^{1-1}}{\log_2 2} + \frac{2^{4-1}}{\log_2 3} + \frac{2^{3-1}}{\log_2 4} = 1 + \frac{15}{1.585} + \frac{7}{2} = 13.96$$

The ideal top 3 is $[W, Y, Z]$:

$$\begin{aligned} \text{IDCG} &= \frac{15}{1} + \frac{7}{1.585} + \frac{3}{2} = 20.92 \\ \text{NDCG} &= \frac{13.96}{20.92} = 0.668 \end{aligned}$$

Drill 4: one ALS round with $d = 1$

$$\begin{aligned} \mathbf{M} &= \begin{pmatrix} 4 & 2 \\ 2 & \cdot \\ \cdot & 6 \end{pmatrix} \\ \mathbf{U} &= \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \end{aligned}$$

Worked answer

- Solve $\mathbf{V}$ ($\mathbf{U}$ fixed):** each prediction is $1 \times v_j$, so v_j = the column's average known rating.
 $v_1 = (4 + 2)/2 = 3, v_2 = (2 + 6)/2 = 4$.
- Solve $\mathbf{U}$ ($\mathbf{V}$ fixed):** with one factor, the least-squares answer is

$$u_i = \frac{\sum_j r_{ij}}{\sum_j v_j^2} \quad \text{(known } j \text{ only)}$$

- Row 1: $(4 \cdot 3 + 2 \cdot 4) / (9 + 16) = 20/25 = 0.8$
 - Row 2: $(2 \cdot 3) / 9 = 0.667$
 - Row 3: $(6 \cdot 4) / 16 = 1.5$
3. **Fill the blanks:** row 2, item 2 = $0.667 \times 4 = 2.67$; row 3, item 1 = $1.5 \times 3 = 4.5$.

Drill 5: coordinate descent with $d = 1$

$$\begin{aligned} \mathbf{M} &= \begin{pmatrix} 3 & 5 \\ 1 & \cdot \end{pmatrix} \\ \mathbf{U} &= \mathbf{V} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \end{aligned}$$

Update u_1 , then u_2 , then v_1 .

Worked answer

1. u_1 : row 1 predicts $u_1 \times 1$ for ratings 3 and 5 $\rightarrow$ best $u_1 = (3 \cdot 1 + 5 \cdot 1)/(1^2 + 1^2) = 4$.
2. u_2 : row 2 only knows item 1 (rating 1, $v_1 = 1$) $\rightarrow u_2 = 1/1 = 1$.
3. v_1 : column 1 knows rows 1 and 2, now with $u = (4, 1)$:

$$v_1 = \frac{3(4) + 1(1)}{4^2 + 1^2} = \frac{13}{17} \approx 0.765$$

With $d = 1$ each person has one number, so CGD's one-number update is the same formula as ALS's one-row update.

MCQ practice (for the 60% part)

Answer all of them first, then check the key at the bottom.

Frequent itemsets and rules

1. Which is always true?
 - A. Adding an item to a group can raise its support.
 - B. Relative support = support $\times$ N.
 - C. $\text{sup}(\{\text{bread}, \text{milk}\}) \leq \text{sup}(\{\text{bread}\})$.
 - D. A group is frequent if its confidence $\geq$ minsup.
2. $\{\text{butter}\}$ is not frequent. Then $\{\text{bread}, \text{butter}\}$ is:
 - A. not frequent
 - B. frequent if bread is frequent
 - C. impossible to tell without counting
 - D. maximal
3. A frequent itemset is **maximal** when:
 - A. it has the highest support
 - B. none of its smaller subsets is frequent
 - C. it contains every item
 - D. none of its bigger supersets is frequent
4. Brute force with d items and N receipts: the work grows
 - A. linearly in d , exponentially in N
 - B. exponentially in d , linearly in N

- C. linearly in both
- D. exponentially in both

5. Apriori's main weakness, which FP-Growth avoids:

- A. it builds candidate groups that may never occur, and rereads the data every level
- B. it can miss frequent itemsets
- C. it cannot use a minimum support
- D. it only finds pairs

6. In FP-Growth, the projected database for item X is:

- A. every receipt, with X crossed out
- B. only receipts without X
- C. only receipts that contain X, keeping just the items after X
- D. the list of frequent itemsets so far

7. Among the three itemset methods, which do the slides call "parallelizable but more complicated to implement"?

- A. Brute force
- B. Apriori
- C. None of them
- D. FP-Growth

8. $\text{conf}(A \rightarrow B)$ equals:

- A. $\text{sup}(A \cup B) / \text{sup}(A)$
- B. $\text{sup}(A \cup B) / \text{sup}(B)$
- C. $\text{sup}(A) / \text{sup}(B)$
- D. $\text{sup}(A \cup B) / N$

9. A rule has confidence 1.0 and lift 1.0. This means:

- A. a strong positive link
- B. a negative link
- C. B always shows up with A, but A doesn't make B any more likely than usual
- D. a calculation error

10. $\text{lift}(A \rightarrow B) = 0.83$ means:

- A. A and B appear together 83% of the time
- B. buying A makes B slightly less likely than average
- C. no link at all

- D. the rule fails minconf

11. Which measure gives the same value if you flip the rule ($A \rightarrow B$ vs $B \rightarrow A$)?

- A. Confidence
- B. Both
- C. Neither
- D. Lift

12. In pyFIM, `fim.ec1at(db, supp=-2)` means:

- A. minsup of 2 receipts
- B. minsup of 2%
- C. itemsets of at most 2 items
- D. return 2 rules

Recommenders and neighborhood CF

13. Counting "clicked the ad" as a like is an example of:

- A. an explicit rating
- B. a content feature
- C. an implicit rating
- D. serendipity

14. "You liked red, round things, so here are more red, round things" is:

- A. user-based CF
- B. content-based
- C. item-based CF
- D. latent factor

15. Why compare two users only on items both rated, instead of filling blanks with 0?

- A. It's faster.
- B. Cosine needs vectors of equal length.
- C. It makes every similarity positive.
- D. A made-up 0 is not a real rating and distorts the similarity.

16. Why subtract each user's average before using cosine?

- A. To remove grading habits, so opposite tastes come out negative
- B. To make all similarities positive
- C. To fill in the blanks

- D. To make it run faster

17. Adjusted cosine is the same as:

- A. raw cosine
- B. Euclidean distance
- C. Pearson correlation over commonly rated items
- D. Jaccard similarity

18. Item-based CF is usually more reliable than user-based because:

- A. there are more items than users
- B. items group neatly (e.g. by genre), while users like several genres
- C. it needs no similarity measure
- D. it skips mean-centering

19. In item-based CF, the predicted rating of user u for item i uses:

- A. other users' ratings of item i
- B. the average of item i
- C. u 's ratings on every item
- D. u 's own ratings on the items most similar to i

Evaluation

20. "How many users can we give n recommendations to?" is which dimension?

- A. Coverage
- B. Accuracy
- C. Diversity
- D. Scalability

21. Offline evaluation:

- A. needs live users
- B. is the same as an A/B test
- C. uses ratings already collected; simplest, but data may be old
- D. costs more than online evaluation

22. In DCG, the item at position 3 is divided by:

- A. 3
- B. $\log_2 4 = 2$
- C. $\log_2 3$

- D. 2^3

23. NDCG = 1 means:

- A. every recommended item was rated the maximum
- B. DCG = 1
- C. nothing was rated
- D. the list is in the ideal order

Latent factors

24. 1,000 users, 500 items, $d = 20$. How many numbers do U and V hold together?

- A. 30,000
- B. 500,000
- C. 10,000
- D. 1,520

25. The error J in latent factor CF adds up squared errors over:

- A. every cell
- B. only the blank cells
- C. only the known ratings
- D. the diagonal

26. ALS compared with SGD:

- A. SGD is steadier
- B. ALS is steadier and parallelizable; SGD is more efficient
- C. ALS needs a step size α
- D. They are identical

27. One step of coordinate gradient descent:

- A. solves a whole table
- B. takes a random-sized step
- C. only ever changes V
- D. solves one number exactly while everything else stays fixed

28. Changing one number in row i of U affects which part of $U V^T$?

- A. Row i only
- B. One column only
- C. The whole matrix

- D. Nothing

29. In Surprise, `trainset.build_anti_testset()` gives:

- A. all known ratings
- B. a random 20% split
- C. every user-item pair with no rating (the ones to predict)
- D. only negative ratings

Answer key

#	Answer	Why
1	C	A bigger group can only be as common as its parts, or rarer.
2	A	Downward closure: if a part fails, anything containing it fails.
3	D	Maximal = frequent, and you can't grow it without failing.
4	B	$2^d - 1$ groups, each checked against N receipts.
5	A	FP-Growth works on shrinking piles and reads the data twice.
6	C	Keep receipts with X; drop X and earlier items.
7	D	Slide summary of the three methods.
8	A	Out of receipts with A, the share that also have B.
9	C	Happens when B is on every receipt (like milk).
10	B	Lift below 1 = negative association.
11	D	$\text{lift} = \frac{\text{sup}(A \cup B) \cdot N}{(\text{sup}(A) \cdot \text{sup}(B))}$, which is symmetric.
12	A	Negative supp = absolute count; positive = percent.
13	C	Inferred from behavior, not given by the user.
14	B	It uses what the items are like.

#	Answer	Why
15	D	Blanks aren't ratings.
16	A	Removes generous vs harsh grading.
17	C	Stated in the notebook.
18	B	One of the two reasons the duality breaks.
19	D	Your own centered ratings on the k most similar items you rated.
20	A	Coverage.
21	C	Offline uses existing data.
22	B	Discount is $\log_2(\text{position} + 1)$.
23	D	DCG equals the ideal DCG.
24	A	$(1,000 + 500) \times 20$.
25	C	Blanks are what we predict.
26	B	From the notebook's ALS and CGD intro.
27	D	One element at a time, in closed form.
28	A	Row i of U only touches row i of the product.
29	C	"Anti" = the pairs not in the training data.

Jupyter part (40%): functions to be able to write

The prof said this part is based on the exercises, so these are the functions to write from memory. Each has a drill with tests in the review notebook.

Function	The one trick to remember	Notebook drill
<code>to_sparse(db)</code>	Columns sorted alphabetically; <code>csr_matrix((data, (rows, cols)), shape=...)</code>	A3

Function	The one trick to remember	Notebook drill
<code>brute(db, minsup)</code>	<code>itertools.combinations</code> for every size, <code>set(t).issuperset(combo)</code>	A4
<code>apriori(db, minsup)</code>	Join two winners sharing their first $k - 2$ items; skip if any $(k - 1)$ -part failed	A6
<code>apriori2(db, minsup)</code>	Keep an itemset only if no other frequent one is a strict superset	A7
<code>fpgrowth(db, minsup)</code>	Recurse on the projected pile: receipts with the item, only the items after it	A9
<code>assoc(fi, minconf, db_size)</code>	One-item consequent; $\text{lift} = \text{conf} \div (\text{sup of consequent} \div N)$; five-part sort	A11
<code>raw_cos(a, b)</code>	Mask with <code>a.notna()</code> & <code>b.notna()</code> ; return 0 if empty or a norm is 0	B2
<code>user_complete(df, k)</code>	<code>df.sub(means, axis=0)</code> ; top k, then keep neighbors who rated the item; weight sum 0 $\rightarrow$ NaN	B4
<code>recommend(user, df, done, n)</code>	Unrated items only; <code>sort_index()</code> then stable <code>sort_values(kind="mergesort")</code>	B5
<code>item_complete(df, k)</code>	Compare columns, but still center by each user's mean	B7
<code>dcg, ndcg</code>	<code>fillna(0)</code> , gain $2^r - 1$, discount $\log_2(\text{rank} + 1)$, average over users	B9
<code>als(M, d, tol)</code>	Start at all ones; <code>lstsq</code> per row; RMSE with <code>np.nanmean</code>	C2
<code>recommend_als(...)</code>	<code>user</code> is a position: <code>f_user[user]</code> and <code>df.iloc[user]</code>	C3
<code>cd(df, k, tol)</code>	All ones; rows of U left to right, then rows of V; stop when $(\text{prev} - \text{now}) \div \text{prev} < \text{tol}$	C5–C6
Surprise	Melt to (user, item, rating); <code>KNNWithMeans</code> ; <code>build_anti_testset()</code> for the blanks	C9

Exam habits

- Copy the problem's sort and tie rule exactly; most failed asserts come from ordering.
- Run the given assert cell right after each function. If it fails, test on a slide table you can check by hand.
- Use only the standard library when the problem says so (the FIM problems do).
- Write the simplest correct version first, then tidy up. Keep lines at 79 characters or fewer.